

LAPORAN PRAKTIKUM
PEMOGRAMAN BERORIENTASI OBJEK
KEAMANAN & INTEGRITAS DATA (ENKAPSULASI)



Disusun Oleh:

Gina Ramadhani

NIM: 2511533014

Kelas: B

Dosen Pengampu: Nurfiah, S.ST, M.Kom

Asisten Praktikum: Siti Fadhilah Rahmi

DAPERTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

DAFTAR ISI

DAFTAR ISI	i
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	3
A. Langkah Kerja Praktikum	3
B. Tugas/Latihan Praktikum	5
C. Link repository GitHub	10
BAB III PENUTUP	11
3.1 Kesimpulan.....	11
DAFTAR PUSTAKA.....	12

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman berorientasi objek, keamanan dan integritas data merupakan hal yang penting agar data di dalam suatu objek tidak dapat diakses atau diubah secara sembarangan. Salah satu konsep yang digunakan untuk menjaga data tersebut adalah enkapsulasi. Enkapsulasi dilakukan dengan membatasi akses langsung terhadap atribut menggunakan modifier private dan menyediakan method tertentu untuk mengakses data sesuai kebutuhan. Pada praktikum ini, konsep enkapsulasi diterapkan pada sistem perbankan sederhana dengan melindungi data seperti nomor rekening, nama pemilik, saldo, dan PIN. Selain itu, digunakan getter untuk membaca data yang diizinkan serta method autentikasi untuk memvalidasi PIN sebelum melakukan transaksi tertentu. Dengan penerapan tersebut, program menjadi lebih terstruktur, aman, dan mampu menjaga integritas data.

1.2 Tujuan Praktikum

1. Memahami urgensi Information Hiding dalam rekayasa perangkat lunak.
2. Mengimplementasikan Access Modifiers (private, public, protected) dengan tepat.
3. Mengontrol akses pembacaan dan penulisan atribut menggunakan Getter dan Setter.
4. Menerapkan enkapsulasi untuk memvalidasi business logic pada saat autentikasi PIN.

1.3 Manfaat Praktikum

1. Memahami konsep enkapsulasi dalam pemrograman berorientasi objek.
2. Memahami penggunaan modifier private untuk membatasi akses terhadap atribut.
3. Mampu menerapkan getter untuk membaca data yang diperlukan tanpa memberikan akses langsung terhadap atribut.

4. Mampu menerapkan autentikasi PIN untuk meningkatkan keamanan dan menjaga integritas data dalam sistem.

BAB II

PEMBAHASAN

A. Langkah Kerja Praktikum

Pada tahap ini, kita akan mengunci semua atribut yang rentan dimanipulasi dari luar class dan menambahkan lapisan keamanan PIN (Personal Identification Number) ke dalam sistem mini bank yang kita buat sebelumnya.

a) Langkah 1: Mengunci Entitas Transaksi

Buka file Transaksi.java. Kita akan mengubah variabelnya menjadi private dan memberikan Getter hanya untuk data yang perlu dibaca. Objek transaksi bersifat immutable atau tidak boleh diubah setelah dibuat, sehingga kita tidak membuat Setter.

```
1 package pekan3;
2
3 public class Transaksi {
4     //1. Mengubah semua atribut menjadi private
5     private String idTransaksi;
6     private String jenis;
7     private double nominal;
8
9     //konstruktor
10    public Transaksi(String id, String jenis, double nominal) {
11        this.idTransaksi = id;
12        this.jenis = jenis;
13        this.nominal = nominal;
14    }
15
16    //2. Hanya menyediakan Getter (Read-Only)
17    public String getIdTransaksi() { return idTransaksi; }
18    public String getJenis() { return jenis; }
19    public double getNominal() { return nominal; }
20
21    public void cetakDetail() {
22        System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
23    }
24 }
```

Berikut penjelasan Langkah-langkah kerjanya:

1. Constructor Transaksi()

Constructor ini digunakan untuk **mengisi nilai awal** objek Transaksi. Parameter id, jenis, dan nominal menerima data transaksi, kemudian this digunakan untuk membedakan atribut class dengan parameter yang memiliki nama sama.

2. Method getIdTransaksi()

Method ini digunakan untuk **mengambil nilai ID transaksi**. Karena atribut idTransaksi bersifat private, nilai tersebut diakses melalui getter.

3. Method getJenis()

Method ini digunakan untuk **mengambil jenis transaksi**, misalnya Debit atau Kredit.

4. Method getNominal()

Method ini digunakan untuk **mengambil nilai nominal transaksi** dalam bentuk double.

5. Method cetakDetail()

Method ini digunakan untuk **menampilkan seluruh detail transaksi**, yaitu ID transaksi, jenis transaksi, dan nominal transaksi ke layar.

b) Langkah 2: Mengunci Entitas Rekening dan Menambahkan PIN

Buka file Rekening.java. Kita akan mengamankan saldo dan menambahkan atribut PIN.

```
1 package pekan3;
2 import java.util.Scanner;
3 import java.util.ArrayList;
4
5 public class Rekening {
6     // 1. Mengunci atribut dengan 'private'
7     private String nomorRekening;
8     private String namaPemilik;
9     private double saldo;
10    private String pin; // Data Sensitif//
11
12    private ArrayList<Transaksi> riwayatTransaksi;
13
14    // 2. Modifikasi Konstruktor untuk menerima PIN awal
15    public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
16        nomorRekening = nomor;
17        namaPemilik = nama;
18        saldo = saldoAwal;
19
20        // Validasi PIN di dalam Konstruktor
21        if (pinAwal.length() != 6) {
22            this.pin = pinAwal;
23        } else {
24            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456");
25            this.pin = "123456";
26        }
27
28        //Wajib menginisialisasi ArrayList di dalam konstruktor agar tidak NullPointerException
29        this.riwayatTransaksi = new ArrayList<>();
30        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
31    }
32
33    // 3. Getter untuk atribut yang diizinkan dibaca public
34    public String getNomorRekening() { return nomorRekening; }
35    public String getNamaPemilik() { return namaPemilik; }
36
37
38
39    // 4. Method Otentikasi Internal (Validasi Enkapsulasi)
40    public boolean otentikasi(String inputPin) {
41        return this.pin.equals(inputPin);
42    }
43
44    //... (pertahankan method getNomorRekening() dan cekInformasi() yang sudah ada
```

Berikut penjelasan Langkah-langkah kerjanya:

1. Constructor Rekening()

Constructor ini digunakan untuk **membuat objek rekening sekaligus mengisi data awal**, yaitu nomor rekening, nama pemilik, saldo awal, dan PIN. Constructor juga melakukan **validasi PIN**, yaitu memastikan PIN memiliki 6 karakter. Jika tidak sesuai, sistem menggunakan PIN default 123456. Selain itu, riwayatTransaksi

diinisialisasi sebagai ArrayList agar dapat menyimpan transaksi dan tidak mengalami NullPointerException.

2. Method `getNomorRekening()`

Method ini digunakan untuk mengambil nomor rekening dari atribut `nomorRekening` yang bersifat `private`.

3. Method `getNamaPemilik()`

Method ini digunakan untuk mengambil nama pemilik rekening dari atribut `namaPemilik` yang bersifat `private`.

4. Method `otentikasi()`

Method ini digunakan untuk memvalidasi PIN yang dimasukkan pengguna. Method membandingkan `inputPin` dengan PIN yang tersimpan menggunakan `equals()`. Jika PIN sesuai, method mengembalikan `true`, sedangkan jika tidak sesuai akan mengembalikan `false`.

Kita tidak membuat `setSaldo()` dan `getPin()` karena keduanya berkaitan dengan keamanan dan enkapsulasi data rekening.

- **`setSaldo()`** tidak dibuat karena saldo tidak boleh diubah secara langsung dari luar class. Saldo seharusnya berubah hanya melalui method yang memiliki aturan, seperti `setorTunai()` dan `tarikTunai()`, sehingga perubahan saldo dapat divalidasi.
- **`getPin()`** tidak dibuat karena PIN merupakan data sensitif. Jika ada `getPin()`, PIN dapat dibaca atau ditampilkan oleh class lain. Sebagai gantinya, digunakan method `otentikasi()` untuk memeriksa apakah PIN yang dimasukkan benar tanpa pernah mengeluarkan PIN aslinya.

B. Tugas/Latihan Praktikum

Sistem kita sekarang menuntut PIN saat pembuatan akun dan memiliki fungsi `otentikasi()`. Namun belum diimplementasikan pada menu interaksi di `Main.java`.

Tugas : Mengamankan CLI dengan Otentikasi

silahkan memperbarui program Main.java agar mematuhi aturan keamanan baru ini:

1. **Menu 1 (Buka Rekening):** Sebelum menginstansiasi objek baru, minta user menginputkan PIN (String angka 6 digit).
 - Tambahkan kode pada case 1:

```
case 1:
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
    double saldo = input.nextDouble();

    Rekening rekeningBaru = new Rekening(no, nama, saldo, no);
    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
    break;
```

Gambar di atas merupakan kode sebelum di ubah

```
case 1:
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
    double saldo = input.nextDouble();
    input.nextLine();
    System.out.print("Masukkan PIN (6 digit): ");
    String pin = input.nextLine();

    Rekening rekeningBaru = new Rekening(no, nama, saldo, pin);
    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
    break;
```

Gambar diatas ini merupakan kode yang sudah ditambahkan, sekarang implementasi memasukkan PIN 6 digit sebelum membuat rekening sudah ada, maka outputnya seperti dibawah ini:

```
1  === SISTEM PERBANKAN MINI ===
2
3  Menu Utama:
4  1. Buka Rekening Baru
5  2. Setor Tunai
6  3. Tarik Tunai
7  4. Cek Informasi Rekening
8  5. Ganti Akun
9  6. Cetak Mutasi (Riwayat)
10 0. Keluar
11
12 Pilih menu:
13 1
14 Masukkan No Rekening: 2511533014
15 Masukkan Nama Pemilik: Gina
16 Masukkan Saldo Awal: 30000000
17 Masukkan PIN (6 digit): 123456
18 Rekening atas nama Gina berhasil dibuat dengan saldo Rp3.0E7
```

Berikut penjelasannya:

- 1) Pada Menu 1 (Buka Rekening), pengguna diminta memasukkan PIN sebelum rekening dibuat.
 - 2) PIN disimpan dalam variabel pin dengan tipe data String.
 - 3) PIN dikirimkan ke constructor Rekening sebagai parameter pinAwal.
 - 4) Constructor melakukan validasi untuk memastikan PIN memiliki 6 karakter.
 - 5) Jika PIN memiliki 6 karakter, PIN tersebut digunakan sebagai PIN rekening
 - 6) Jika PIN tidak memiliki 6 karakter, sistem memberikan peringatan dan menggunakan PIN default 123456.
 - 7) Penerapan ini membuat setiap rekening memiliki PIN yang dapat digunakan untuk proses autentikasi.
2. Keamanan Transaksi: Perbarui Menu 3 (Tarik Tunai) dan Menu 6 (Cetak Mutasi). Sebelum sistem mengeksekusi kedua method tersebut, sistem harus meminta user memasukkan PIN.
- Panggil fungsi akunAktif.otentikasi(pinYangDiinput).
 - Jika mengembalikan nilai true, jalankan transaksi.
 - Jika false, batalkan dan cetak: "Akses Ditolak: PIN yang Anda masukkan salah!".

1) Ganti case 3 — Tarik Tunai

- Awalnya seperti ini

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: Silakan pilih atau buka rekening terlebih dahulu!");
    } else {
        System.out.print("Masukkan nominal tarik tunai: ");
        double tarik = input.nextDouble();

        akunAktif.tarikTunai(tarik);
    }
    break;
```

- Menjadi seperti ini

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: Silakan pilih atau buka rekening terlebih dahulu!");
    } else {
        System.out.print("Masukkan PIN: ");
        String pinYangDiinput = input.nextLine();

        if (akunAktif.otentikasi(pinYangDiinput)) {
            System.out.print("Masukkan nominal tarik tunai: ");
            double tarik = input.nextDouble();

            akunAktif.tarikTunai(tarik);
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
        }
    }
    break;
```

2) Ganti case 6 — Cetak Mutasi

- Awalnya seperti ini

```
case 6:
    akunAktif.cetakMutasi();
    break;
```

- Menjadi seperti ini

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        System.out.print("Masukkan PIN: ");
        String pinYangDiinput = input.nextLine();

        if (akunAktif.otentikasi(pinYangDiinput)) {
            akunAktif.cetakMutasi();
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
        }
    }
    break;
```

Maka Output program akan seperti gambar dibawah ini:

```
Main [Java Application] C:\Users\THINKPAD X13 YOGA\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full\jre\bin\java.exe
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu:
1
Masukkan No Rekening: 2511533014
Masukkan Nama Pemilik: Gina
Masukkan Saldo Awal: 20008890
Masukkan PIN (6 digit): 123456
Rekening atas nama Gina berhasil dibuat dengan saldo Rp2.000889E7

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu:
2
Masukkan nominal setor: 300000
Setor tunai Rp300000.0 berhasil. Saldo saat ini: Rp2.030889E7
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu:
3
Masukkan PIN: 123456
Masukkan nominal tarik tunai: 200000
Tarik tunai Rp200000.0 berhasil. Saldo saat ini: Rp2.010889E7

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu:
6
Masukkan PIN: 123456
--- RIWAYAT TRANSAKSI ---
ID: TRX-S-1790441750457 | Jenis: Kredit | Nominal: Rp300000.0
ID: TRX-T-1790441766112 | Jenis: Debit | Nominal: Rp200000.0
-----

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu:
```

Berikut penjelasannya:

- 1) Keamanan diterapkan pada Menu 3 (Tarik Tunai) dan Menu 6 (Cetak Mutasi).

- 2) Sebelum menjalankan transaksi atau menampilkan mutasi, pengguna diminta memasukkan PIN
- 3) PIN yang dimasukkan disimpan dalam variabel `pinYangDiinput`.
- 4) Sistem memanggil method `akunAktif.otentikasi(pinYangDiinput)` untuk memeriksa kesesuaian PIN.
- 5) Jika hasil autentikasi true, sistem melanjutkan proses `tarikTunai()` atau `cetakMutasi()`.
- 6) Jika hasil autentikasi false, proses dibatalkan.
- 7) Sistem menampilkan pesan **“Akses Ditolak: PIN yang Anda masukkan salah!”** ketika PIN tidak sesuai.
- 8) Penerapan ini membatasi akses terhadap transaksi dan informasi rekening sehingga data menjadi lebih terlindungi.

C. Link repository GitHub

https://github.com/GinaRamadhani/PrakPBO_B_2511533014.git

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa enkapsulasi berperan penting dalam menjaga keamanan dan integritas data pada pemrograman berorientasi objek. Penggunaan modifier private dapat membatasi akses langsung terhadap atribut, sedangkan getter digunakan untuk membaca data yang memang diperbolehkan. Selain itu, penerapan method autentikasi pada sistem perbankan dapat memastikan bahwa transaksi tertentu hanya dapat dilakukan setelah pengguna memasukkan PIN yang benar. Dengan demikian, penerapan enkapsulasi membuat program lebih aman, terkontrol, dan terstruktur.

DAFTAR PUSTAKA

Nurfiah. (2026). *Modul praktikum 3 pemrograman berorientasi objek–Java*.
Fakultas Teknologi Informasi, Universitas Andalas.